

BOOK MANAGEMENT SYSTEM (BMS)

Technical Documentation

1. Project Overview

The Book Management System (BMS) is a full-stack application used to manage books, users, and administrative workflows such as approvals, role management, and inventory control.

Technologies Used:

- **Frontend:** React (TypeScript)
 - **Backend:** NestJS
 - **Database:** PostgreSQL (TypeORM)
 - **Authentication:** JWT
 - **API Type:** RESTful API
-

2. System Architecture

The system follows a **3-layer architecture**:

2.1 Frontend Layer

- React-based UI
- Handles user interactions
- Communicates with backend via Axios

2.2 Backend Layer

- NestJS API server
- Handles business logic
- Authentication & authorization
- Database operations

2.3 Database Layer

- PostgreSQL

- Stores users, books, roles, and statuses
-

3. Backend Structure (NestJS)

3.1 Project Folder Structure

```
src/  
|  
| └─ main.ts  
| └─ app.module.ts  
|  
| └─ entities/  
|   | └─ user.entity.ts  
|   | └─ book.entity.ts  
|  
| └─ users/  
|   | └─ users.controller.ts  
|   | └─ users.service.ts  
|  
| └─ books/  
|   | └─ books.controller.ts  
|   | └─ books.service.ts  
|  
| └─ admin/  
|   | └─ admin.controller.ts  
|   | └─ admin.service.ts  
|  
| └─ auth/  
|   | └─ auth.controller.ts  
|   | └─ auth.service.ts  
|  
| └─ dto/  
|   | └─ updateStatusDto.ts  
|  
| └─ common/  
|   | └─ guards/  
|   |   | └─ jwt-auth.guard.ts
```

```
| | └─ roles.guard.ts
| └─ decorators/
| └─ roles.decorator.ts
```

3.2 Module Structure

App Module

- Root module that imports all feature modules

Users Module

- Handles user registration
- User approval workflow
- Role management

Books Module

- CRUD operations for books

Auth Module

- Login / JWT generation
- Password validation

Admin Module

- Admin-specific operations:
 - User approval
 - User promotion
 - Book management
-

4. Database Schema

4.1 User Table

Field	Type	Description
id	UUID	Primary Key

Field	Type	Description
name	string	Full name
email	string	Unique
password_hash	string	Encrypted password
role	enum	admin / user
status	enum	pending / active / rejected
createdAt	timestamp	Created time
updatedAt	timestamp	Updated time

4.2 Book Table

Field	Type	Description
id	UUID	Primary Key
title	string	Book title
author	string	Book author
description	string	Book details
publishedYear	number	Year published

5. Backend API Endpoints

5.1 User APIs

GET /users
 GET /users/status/:status
 GET /users/pending
 GET /users/:id
 POST /users
 PATCH /users/:id/status
 PATCH /users/:id/reject

PATCH /users/:id/approve
DELETE /users/:id

5.2 Book APIs

GET /books
POST /books
PATCH /books/:id
DELETE /books/:id

5.3 Admin APIs

GET /admin/users
PATCH /admin/user/approve/:id
PATCH /admin/user/promote/:id
DELETE /admin/user/:id

GET /admin/books
POST /admin/book
PATCH /admin/book/:id
DELETE /admin/book/:id

6. Authentication System

- Uses JWT tokens
- Passwords hashed using bcrypt

Flow:

1. User logs in
2. Server validates credentials
3. JWT token generated
4. Token stored in frontend
5. Sent in Authorization header

Authorization: Bearer <token>

7. Role-Based Access Control (RBAC)

Role Permissions

Admin Full system access

User View + limited actions

8. Frontend Structure

```
src/  
├── pages/  
|   ├── AdminUsers.tsx  
|   ├── AdminBooks.tsx  
|   ├── Login.tsx  
|   ├── Dashboard.tsx  
|  
├── services/  
|   ├── api.ts  
|  
├── components/  
|   ├── Navbar.tsx  
|   ├── Sidebar.tsx
```

9. Key Features

- User registration & login
- Admin approval system
- Role promotion
- Book management (CRUD)
- Status tracking (pending / active / rejected)
- Protected routes (JWT + Guards)
- Swagger API documentation

10. Swagger Documentation

Available at:

<http://localhost:3000/api>

11. Known Issues Fixed

- Fixed UUID vs number mismatch
 - Fixed 404 API routing issues
 - Fixed CORS configuration
 - Fixed React duplicate fetch logic
 - Fixed approve/reject workflow
-

12. Future Improvements

- Pagination for users/books
 - Search functionality
 - Analytics dashboard
 - Email notifications
 - Audit logs for admin actions
-

 **END OF DOCUMENT**